

BRDS-PEC: Behavioral Ransomware Detection System

Zero-to-Hero Mastery & Reviewer Defense Guide (Complete Explanation Package)

1. The Core Concept: How to Explain BRDS-PEC in 60 Seconds

Imagine a bank security guard. Traditional Antivirus (AV) is like a guard holding a photo album of known criminals (static file hashes). If a burglar puts on a new mask or changes their clothes (zero-day obfuscation), the static guard lets them walk right in. **BRDS-PEC** is a smart AI guard that doesn't care what the burglar looks like—it watches what they *do*. If a process starts carrying a crowbar, smashing windows (modifying files), deleting key backups (vssadmin shadow copy wipe), and preparing to lock the vault doors, BRDS-PEC tackles the burglar and isolates the room within 5 to 15 seconds—**before the vault doors are locked (pre-encryption containment)**.

2. Why Traditional EDR & Antivirus Fail

- **Static Hash Evasion:** Attackers re-pack binaries or modify single bits, completely bypassing Signature AV.
- **Speed of Ransomware:** Ransomware encrypts thousands of files per minute and wipes Windows Shadow Copies (vssadmin delete shadows /all /quiet) in under 10 seconds.
- **Post-Encryption Alerting:** Most commercial tools trigger alerts AFTER ransom notes appear, resulting in total data loss.
- **BRDS-PEC Advantage:** Operates on 5-second sliding Sysmon behavioral windows using a PyTorch Bidirectional LSTM to halt execution in the *pre-encryption phase*.

3. End-to-End System Architecture (The 7-Step Pipeline)

```
STEP 1: Windows Sysmon Logging (Event IDs 1, 3, 7, 11, 12, 13, 23, 26)
↓
STEP 2: Temporal Aggregation (5-second sliding UTC windows -> 17 numeric features)
↓
STEP 3: Tier 1 Baseline Screening (Logistic Regression + Isolation Forest, risk >= 0.50)
↓
STEP 4: Tier 2 PyTorch LSTM Sequence Classifier (30-step sequence history evaluation)
↓
STEP 5: Cryptographic HMAC-SHA256 Alert Container Signing & Arm Token Creation
↓
STEP 6: Host Containment (ContainHost.ps1 NIC disable + kill_process_tree.ps1 OS denylist)
↓
STEP 7: Live SOC Dashboard Rendering & PyTorch Autograd SHAP XAI PDF Generation
```

4. The 17 Integrated Behavioral Telemetry Features

BRDS-PEC evaluates 17 quantitative behavioral features extracted every 5 seconds for every running process:

Feature Name	Sysmon Event IDs	Behavioral Significance
event_count	All Events	Total event volume in 5s. Spikes during automated attack execution.
unique_images	Event ID 1	Number of distinct executable binary paths spawned.
unique_files	IDs 11, 23, 26	Count of distinct target files created/modified. Spikes during encryption.
unique_extensions	Event ID 11	Count of new file extensions appended (.WNCRY, .lockbit, .basta).
unique_destination_ips	Event ID 3	Outbound IP connections for C2 beaconing or SMB network spread.
suspicious_path_count	Event ID 1	Executions originating from Temp, AppData, Public, or Downloads.
file_activity_count	IDs 11, 23, 26	Combined file system operation count (create, delete, wipe).
registry_activity_count	IDs 12, 13	Registry edits for persistence (Run keys) or disabling Defender.
network_activity_count	Event ID 3	Network socket creation count.
event_1..26_count	1,3,7,11,12,13,23,26	Individual per-Event-ID frequency counters.

5. Codebase File Map & Module Responsibilities

- **pipeline/temporal_aggregator.py:** Core 5s sliding windowing engine. Groups Sysmon records by computer, process key, and UTC window.
- **ml_engine/lstm/model.py:** 2-Layer Bidirectional LSTM architecture with concatenated Mean + Max pooling.
- **ml_engine/lstm/infer.py:** Hardened model inference engine with SHA-256 hash checks and `weights\_only=True` loading.
- **containment/alert_integrity.py:** HMAC-SHA256 container signing and single-use `arm\_token` generation.
- **containment/kill_process_tree.ps1:** Bottom-up process tree collapse with `\$PROTECTED\_PROCESSES` OS denylist (`lsass`, `svchost`, `explorer`).
- **backend/routes/xai_routes.py:** Exposes REST API endpoints for PyTorch autograd feature attributions and downloadable PDF reports.
- **frontend/index.html & js/:** Dark-mode SOC dashboard featuring Chart.js rolling risk curves and real-time incident cards.

6. Empirical Benchmark & Validation Results

Evaluated on **20,402 behavioral windows** (17,617 genuine Windows 11 benign windows from SILRAD-1.0 + 2,785 attack windows from Splunk ATT&CK; telemetry):

Metric	Baseline & LSTM Score	Evaluation Meaning
--------	-----------------------	--------------------

F1 Score	99.51%	Harmonic mean of precision and recall on source-level split.
Precision	99.53%	High alert quality with minimal false alarms.
Recall	99.48% (100% on LSTM)	Near-zero missed attacks across all ransomware scenarios.
False Positive Rate	0.22%	Operational stability on clean Windows 11 endpoint baseline.
Held-Out F1 (Unseen Attacks)	93.14% Average	Proven generalization to ransomware families never seen in training.

7. Master Reviewer Q&A; Cheat Sheet (Top 10 Defense Questions)

Q1: How does BRDS-PEC differ from traditional EDR or Antivirus?

Answer: Antivirus uses static file hashes easily bypassed by re-packing. EDRs often react after files are encrypted. BRDS-PEC evaluates sliding 5-second Sysmon behavioral windows to halt execution in the pre-encryption phase.

Q2: Why use a 2-Tier Machine Learning architecture?

Answer: Tier 1 (Logistic Regression + Isolation Forest) rapidly screens tens of thousands of background events with sub-millisecond latency. Tier 2 (PyTorch Bidirectional LSTM) evaluates 30-step process history only for flagged anomalies, balancing speed and deep sequence accuracy.

Q3: How do you prove the system detects unseen ransomware families?

Answer: We conducted Leave-One-Scenario-Out Cross-Validation (``scenario_holdout_eval()``). The model achieved 93.14% Average F1 on attack scenarios completely held out from training.

Q4: How do you prevent auto-containment from causing Blue Screen of Death (BSOD)?

Answer: ``kill_process_tree.ps1`` checks a ``$PROTECTED_PROCESSES`` denylist (``lsass``, ``csrss``, ``svchost``, ``explorer``, ``services``) before process tree termination.

Q5: What stops malware from forging fake security alerts?

Answer: ``containment/alert_integrity.py`` signs all alert containers using HMAC-SHA256 and issues single-use ``arm_token`` authorization files.

Q6: How is the PyTorch model protected against deserialization RCE attacks?

Answer: ``ml_engine/lstm/infer.py`` loads model checkpoints using ``torch.load(weights_only=True)`` and verifies the SHA-256 hash manifest before loading.

Q7: What features spike during a ransomware attack?

Answer: ``unique_files``, ``unique_extensions`` (`WNCRY`, `lockbit`, `basta`), ``file_activity_count``, and ``event_1_count`` (spawning ``vssadmin`` or ``cmd.exe``).

Q8: Why does Analyzed Events start at 24,000+ on the dashboard?

Answer: An enterprise endpoint agent processes background events continuously. The counter reflects cumulative historical events stored in the ``brds.db`` database.

Q9: How does the Explainable AI (XAI) feature work?

Answer: It uses PyTorch Autograd integrated gradients ($|\nabla_x y * x|$) to compute exact numerical feature contributions, which can be downloaded as a PDF report.

Q10: Is there data loss during auto-containment?

Answer: No. Because containment executes in the pre-encryption phase (5-15s), user files on disk remain unencrypted. Only memory buffers in the killed malicious process tree are terminated.

8. Top 5 Core Presentation Code Snippets

Include these key code blocks in your presentation slides to demonstrate technical depth:

Snippet 1: PyTorch Bidirectional LSTM Model (ml_engine/lstm/model.py)

```
class LSTMClassifier(nn.Module):
    def __init__(self, input_dim: int = 17, hidden_dim: int = 64, num_layers: int = 2):
        super().__init__()
        self.input_projection = nn.Linear(input_dim, hidden_dim)
        self.lstm = nn.LSTM(hidden_dim, hidden_dim, num_layers=num_layers, batch_first=True,
bidirectional=True)
        self.fc = nn.Linear(hidden_dim * 4, 1) # Mean + Max pooling output
        self.sigmoid = nn.Sigmoid()

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        projected = self.input_projection(x)
        lstm_out, _ = self.lstm(projected)
        mean_pool = lstm_out.mean(dim=1)
        max_pool = lstm_out.max(dim=1).values
        pooled = torch.cat([mean_pool, max_pool], dim=1)
        return self.sigmoid(self.fc(pooled))
```

Snippet 2: RCE-Hardened Model Checkpoint Verification (ml_engine/lstm/infer.py)

```
def load_checkpoint(model_path: Path):
    # Verify SHA-256 checksum manifest before loading
    expected_hash = model_path.with_suffix('.sha256').read_text().strip()
    actual_hash = hashlib.sha256(model_path.read_bytes()).hexdigest()
    if not hmac.compare_digest(expected_hash, actual_hash):
        raise ValueError('Model SHA-256 checksum mismatch (tampered model)')
    # RCE Protection: load weights only (disallows arbitrary pickle code)
    checkpoint = torch.load(model_path, weights_only=True)
    scaler = joblib.load(model_path.with_suffix('.scaler.joblib'))
    return checkpoint, scaler
```

Snippet 3: Cryptographic HMAC-SHA256 Alert Container Signing (containment/alert_integrity.py)

```
def sign_alerts(alerts: list) -> str:
    payload_str = json.dumps(alerts, separators=(',', ':'), sort_keys=True)
    key = os.environ.get('BRDS_ALERT_HMAC_KEY').encode('utf-8')
    signature = hmac.new(key, payload_str.encode('utf-8'), hashlib.sha256).hexdigest()
    return json.dumps({'alerts': alerts, 'sig': signature}, indent=2)
```

Snippet 4: OS Protected Process Denylist & Containment Safety (containment/kill_process_tree.ps1)

```
# Protected Windows critical OS processes
$PROTECTED_PROCESSES = @('lsass', 'csrss', 'smss', 'wininit', 'winlogon', 'services', 'system',
'svchost', 'explorer', 'spoolsv', 'dwm')

foreach ($child in $children) {
    if ($PROTECTED_PROCESSES -notcontains $child.Name.Replace('.exe','')) {
        Stop-ProcessTree -targetPid $child.ProcessId # Recurse child tree
        Stop-Process -Id $child.ProcessId -Force # Terminate ransomware process
    }
}
```

Snippet 5: PyTorch Autograd Integrated Gradient XAI (ml_engine/xai/shap_explainer.py)

```
self.lstm_infer.model.zero_grad()
input_tensor = torch.tensor(features_scaled, dtype=torch.float32).unsqueeze(0).requires_grad_(True)
output = self.lstm_infer.model(input_tensor)
output.backward()

grads = input_tensor.grad.detach().numpy()[0]
grad_x_input = np.abs(grads * features_scaled).mean(axis=0)
attributions = [{'feature_name': name, 'importance_value': float(val)} for name, val in
zip(self.lstm_infer.feature_names, grad_x_input)]
```

9. Step-by-Step Presentation & Live Demo Playbook

Follow this sequence during your live presentation to wowed evaluators:

1. **Start Server:** Execute `$env:BRDS_ALLOW_INSECURE_DEV_HMAC='1'; python backend/app.py`.
2. **Open Dashboard:** Navigate to `http://127.0.0.1:5000/` in Chrome.
3. **Highlight Baseline:** Show Analyzed Events (20,402+ windows) and the green low risk score timeline.
4. **Demonstrate Threat Detection:** Scroll to Active Security Incidents, showing ransomware cards (BLACKBASTA, WANNACRY, LOCKBIT) with 0.85+ risk score.
5. **Explain XAI & Download PDF:** Click `SHAP Analysis` on an incident card, show the positive feature attributions (file activity & extension spriting), and click `Download PDF Report` to show the generated ReportLab document.
6. **Verify Automated Tests:** Run `python -m pytest -v` in the terminal to show `**30 / 30 passing unit tests**`.